

Section 1: Introduction, purpose, hardware, design.....	3
SAIL Hardware.....	3
SAIL Storage Design.....	3
SAIL Network Design.....	4
SAIL Operational Goals.....	4
Section 2: Software choices and architecture decisions.....	5
Architecture Overview.....	5
Section 3: Operating system setup.....	5
Operating System Installation.....	6
Section 4: Physical setup and wiring.....	10
Network Topology Diagram.....	10
Hardware Layout.....	11
Hardware Wiring.....	11
Ethernet Cable Construction.....	12
Physical Environment Assembly.....	14
Section 5: Network Configuration.....	15
Network Planning and IP Address Allocation.....	15
Planned DHCP Reservations.....	17
IP Address Verification.....	18
Environment Shutdown Procedure.....	18
Section 6: Kubernetes setup.....	19
Kubernetes Installation.....	19
Section 7: Workload Design.....	21
Application Workload.....	22
Security Workload.....	22
Automated/AI Workload.....	23
Operational Workload.....	23
Integrated Workload Flow.....	24
Section 8: Application Development and Deployment.....	24
Application Design.....	24
Application Structure.....	26
Application Behavior.....	27
Application Deployment.....	27
Create the Application Environment.....	28
Step 1: Create the application folder and virtual environment.....	28
Step 2: Configure VS Code to Use the Environment.....	28
Step 3: Configure Git for the Application.....	28

Step 4: Install Application and Development Dependencies.....	30
Step 5: Create the Application Project Structure.....	30
Step 6: Configure Application Settings and Environment Variables.....	32
Step 7: Configure PostgreSQL for Development.....	33
Step 8: Configure SQLAlchemy and database connectivity.....	34
Step 9: Configure Alembic database migrations.....	35
Step 10: Implement the user data model and database schema.....	36
Section 9: Monitoring, Logging, & Observability.....	40
Section 10: Security & Infrastructure Automation.....	40
Section 11: AI-Assisted Operations & Deployment Automation.....	41
Section 12: Event Scenarios and Investigation Workflows.....	41

Section 1: Introduction, purpose, hardware, design

SAIL, which stands for Security and AI Infrastructure Lab, is a self hosted infrastructure and automation environment designed to support hands-on learning in DevOps, cybersecurity, artificial intelligence, and systems operations. The environment uses Raspberry Pi systems and an Apple Silicon computer connected through a local home network to create a small scale computing environment where infrastructure, monitoring, automation, and AI related systems can be configured and operated together.

The purpose of the environment is to provide hands-on experience with deployment, monitoring, troubleshooting, automation, logging, security tooling, and operational maintenance through the use of real hardware and locally hosted services. Multiple systems and services run across the shared network, including containerized applications, monitoring tools, log collection systems, AI related services, and automation workflows. The environment also allows operational failures, alert generation, troubleshooting activities, and recovery procedures to be tested and documented over time. All systems connect through a local wired home network using Ethernet connections. The environment runs on existing hardware using free and locally hosted software and services.

SAIL Hardware

SAIL uses the following hardware:

Device	Purpose
Raspberry Pi 4 8GB x3	Kubernetes cluster nodes
MacBook	Administration workstation, AI services, documentation, and automation tools
1TB External SSD	Persistent storage for cluster data, logs, metrics, backups, and documentation assets
Ethernet cables	Network connectivity between systems
Network switch	Local wired network connection between devices

SAIL Storage Design

SAIL uses a single 1TB external SSD as centralized storage for the environment. The storage drive connects to pi-node-1 and stores long term operational and project data generated by the environment.

The centralized storage supports:

- Monitoring data
- Logs
- Kubernetes persistent storage
- AI model files
- Documentation

- Screenshots and diagrams
- Backup files
- Project assets

The Raspberry Pi systems maintain their own operating system storage while the external SSD attached to pi-node-1 stores larger persistent datasets and operational records generated by the environment.

SAIL Network Design

All SAIL systems operate on a local wired network inside the home environment. Each device connects through Ethernet to improve stability and simplify troubleshooting.

The systems use the following names:

System	Role
MacBook	Admin workstation, AI host, documentation, kubectl, Ansible, Git
pi-node-1	k3s control plane, schedulable worker node, and external storage host
pi-node-2	k3s worker
pi-node-3	k3s worker

SAIL Operational Goals

The primary purpose of SAIL is to provide a controlled environment for developing and evaluating operational, security, and AI-assisted investigation workflows.

The environment is intended to support:

- Kubernetes administration and operations
- Infrastructure monitoring and observability
- Centralized logging and event collection
- Runtime security monitoring
- Vulnerability assessment
- Configuration automation
- Failure testing and recovery exercises
- AI-assisted analysis of operational and security events
- Hands-on DevOps and cybersecurity learning

Rather than serving only as a Kubernetes cluster, SAIL functions as a platform for generating, detecting, investigating, and responding to infrastructure and security events. The environment allows operational failures, configuration issues, security alerts, and other events to be intentionally introduced and analyzed using monitoring, logging, security, and AI tooling.

The long-term objective is to develop repeatable workflows that demonstrate how modern infrastructure, security operations, and AI-assisted analysis can be combined within a self-hosted environment.

Section 2: Software choices and architecture decisions

Technology	Purpose	Reason for Selection
k3s	Kubernetes cluster management	Lightweight and well suited for Raspberry Pi systems
Ollama	Local AI model hosting	Runs AI models locally on Apple Silicon hardware
Open WebUI	AI web interface	Browser based access to local AI models
Prometheus	Metrics collection	Tracks system and service health
Grafana	Monitoring dashboards	Visualizes metrics and operational data
Loki	Log collection	Centralized log storage and searching
Falco	Runtime security monitoring	Detects suspicious activity in containers and Linux systems
Trivy	Vulnerability scanning	Scans containers and configurations for known issues
Ansible	System configuration automation	Reduces repetitive setup work across systems
Docker	Container runtime	Runs services in isolated environments
GitHub	Documentation and configuration tracking	Stores notes, configurations, diagrams, and troubleshooting records

Architecture Overview

The environment is organized around a Kubernetes cluster running on three Raspberry Pi systems. One Raspberry Pi serves as the k3s control plane and also participates as a worker node, while the remaining Raspberry Pi systems serve as worker nodes. Kubernetes schedules workloads across the available cluster nodes based on resource availability and configuration. The MacBook functions as an administration workstation and hosts AI related services, documentation assets, and management tools used to operate the environment. The environment uses a local wired network and centralized storage provided through an external SSD connected to pi-node-1. All services operate locally using existing hardware and free software.

Section 3: Operating system setup

The SAIL environment uses Raspberry Pi OS Lite (64-bit) on all Raspberry Pi systems. Raspberry Pi OS was selected because it is the operating system maintained specifically for Raspberry Pi hardware and provides broad compatibility with the tools planned for the environment, including k3s, Prometheus, Grafana, Loki, Falco, Trivy, Docker, and Ansible.

During the planning phase, alternative operating systems were evaluated, including Ubuntu Server. Both options appeared capable of supporting the planned architecture.

Raspberry Pi OS was ultimately selected because the primary objective of SAIL is to provide a stable platform for infrastructure, security, monitoring, automation, and AI-assisted investigation workflows. As the operating system designed specifically for the hardware used in the project, Raspberry Pi OS offers strong hardware support, extensive community documentation, lower resource overhead, and broad compatibility with the planned software stack.

Operating System Installation

The following are performed from a computer connected to the SAIL environment.

Step 1: Download Raspberry Pi Imager

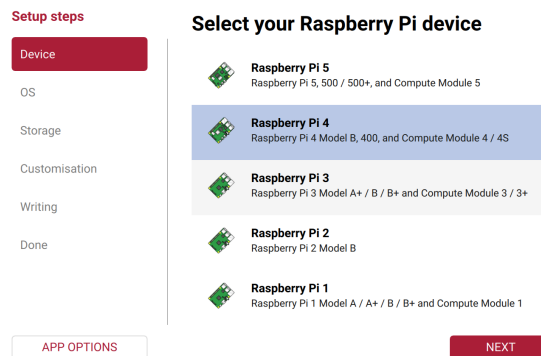
1. Open a web browser. Navigate to: <https://www.raspberrypi.com/software/>
2. Download Raspberry Pi Imager.
3. Install the application.
4. Launch Raspberry Pi Imager.

Step 2: Prepare the First MicroSD Card

1. Insert a MicroSD card into the computer.
2. Verify the operating system detects the card before proceeding.

Step 3: Select the Raspberry Pi Device

1. Within Raspberry Pi Imager:
 - a. Select **Device**
 - b. Select **Raspberry Pi 4**



- c. Choose **NEXT**

Step 4: Select the Operating System

1. Select **Raspberry Pi OS (other)**

Choose operating system

Select an operating system to install on your Raspberry Pi



Raspberry Pi OS (Legacy, 32-bit)

A port of Debian Bookworm with security updates and desktop environment
Online - 1.2 GB download
Released: 2026-06-18



Raspberry Pi OS (other)

Other Raspberry Pi OS based images

2. Select Raspberry Pi OS Lite (64-bit)

Choose operating system

Select an operating system to install on your Raspberry Pi



Back

Go back to main menu



Raspberry Pi OS Lite (64-bit)

A port of Debian Trixie with no desktop environment
(Compatible with Raspberry Pi 3/4/400/5)
Online - 500.6 MB download
Released: 2026-06-18



Raspberry Pi OS Full (64-bit)

A port of Debian Trixie with desktop environment and recommended applications
Online - 1.9 GB download
Released: 2026-06-18

Step 5: Select Storage

1. Select the inserted MicroSD card



Apple SDXC Reader Media

29.2 GB

Mounted as /Volumes/NO NAME

Step 6: Configure Operating System Customization

1. Enter Hostname: pi-node-1
2. Enter Capital city, Time zone, and keyboard layout

OS

Storage

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

Writing

Customisation: Localisation

Select your location for suggested time zone and keyboard layout

Capital city: Washington, D.C. (United States) ⓘ

Time zone: America/Los_Angeles

Keyboard layout: us

3. Username: <your username>

4. Password: <your password>

Customisation: Choose username

Create a user account for your Raspberry Pi

Username:

Password:

Confirm password:

The username must be lowercase and contain only letters, numbers, underscores, and hyphens.

5. Click on Remote Access

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

6. Enable: SSH

Customisation: SSH authentication

Configure SSH access

Enable SSH

[Learn about SSH](#)



Authentication mechanism:

☒ Use password authentication

☐ Use public key authentication

7. Click writing, confirm settings and click **WRITE**

Storage

Customisation

Hostname

Localisation

User

Wi-Fi

Remote access

Raspberry Pi Connect

Writing

Done

Write image

Review your choices and write the image to the storage device

Summary

Device: Raspberry Pi 4

Operating system: Raspberry Pi OS Lite (64-bit)

Storage: Apple SDXC Reader Media

Customisations to apply:

- SSH enabled

APP OPTIONS

BACK

WRITE

Step 7: Write the Image

1. Wait for image creation and verification to complete.
2. Label the MicroSD card: pi-node-1

Step 8: Create Remaining Node Images

1. Repeat Steps 2 through 7.
2. For the second card use: Hostname: pi-node-2
3. For the third card use: Hostname: pi-node-3
4. Label each completed MicroSD card.

Step 9: Install the MicroSD Cards

1. Insert the appropriate MicroSD card into each Raspberry Pi.
2. Verify that each Raspberry Pi receives the card matching its hostname.

Step 10: Boot the Cluster

1. Verify:
 - a. Ethernet cables are connected
 - b. The SSD is connected to pi-node-1
 - c. Power supplies are connected
2. Apply power to the Raspberry Pi systems.
3. Allow several minutes for the initial boot process to complete.

Step 11: Identify Assigned IP Addresses

1. Log in to the router or gateway administration interface.
2. Locate the Raspberry Pi systems in the connected devices list.
3. Record for each node:
 - a. Hostname
 - b. MAC Address
 - c. IP Address

Step 12: Verify SSH Connectivity

1. Open a terminal on the computer.
2. Connect to pi-node-1: `ssh <username>@<ip-address>`
 - a. Example: `ssh sailadmin@10.0.0.20`
3. When prompted: Are you sure you want to continue connecting (yes/no/[fingerprint])?
 - a. Type: yes
4. Enter the password configured during imaging.
5. Verify a successful login.
6. Repeat for pi-node-2 and pi-node-3.

Step 13: Update Raspberry Pi OS

1. Run: `sudo apt update`
`sudo apt full-upgrade -y`
2. Wait for updates to complete.

Step 14: Reboot the Node

1. Run: `sudo reboot`
2. Wait approximately one minute.
3. Reconnect using SSH.
4. Repeat Steps 13 and 14 on all three nodes.

Step 15: Verify Hostname Configuration

1. Run: `hostname`
2. Verify the hostname matches the intended node name.

Step 16: Verify Operating System Installation

1. Run: `cat /etc/os-release`
2. Verify that Raspberry Pi OS Lite (64-bit) is installed.

Step 17: Record Baseline System Information

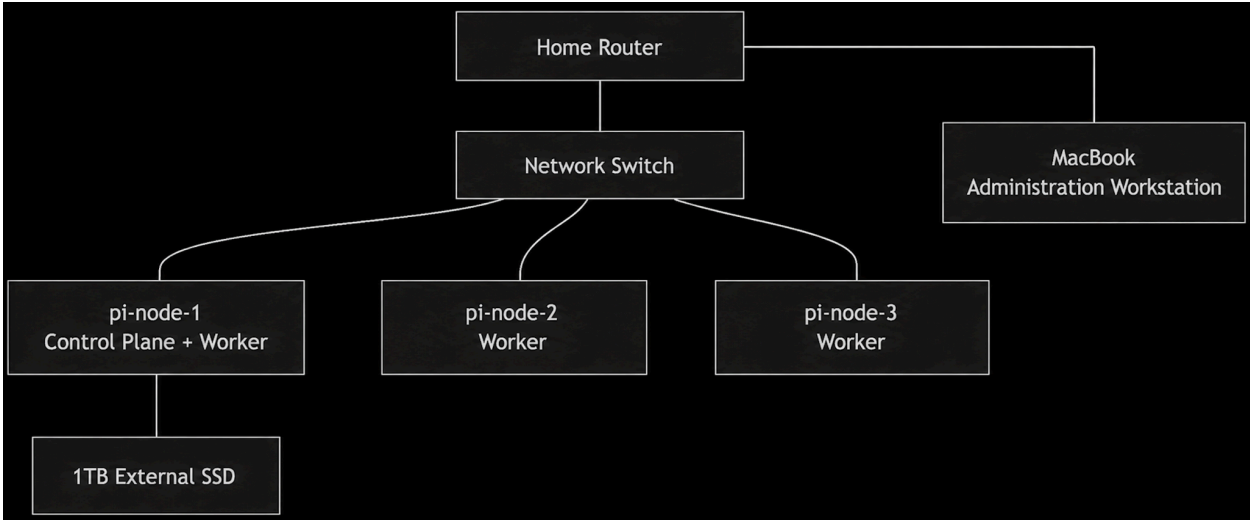
1. Run: `free -h`
2. Record available memory.
3. Run: `df -h`
4. Record available storage. This information is used later when evaluating cluster resource utilization and monitoring data.

Operating system installation is complete when:

- All three Raspberry Pi systems boot successfully
- SSH access is verified
- Operating system updates are installed
- Hostnames are configured correctly
- Network connectivity is verified
- Baseline system information has been recorded

Section 4: Physical setup and wiring

Network Topology Diagram



Hardware Layout

Physical Tower Layout
Top

Netgear GS105E

pi-node-1
pi-node-2
pi-node-3
SSD hard drive Anker Charger

Bottom

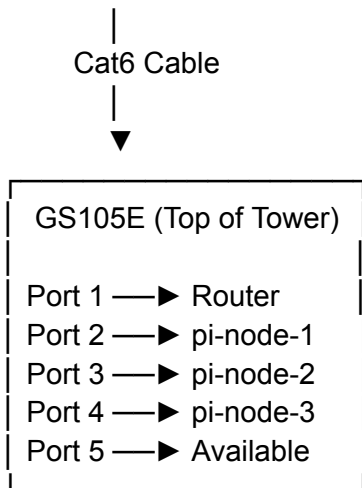
Rear

Fan 1
Fan 2

Hardware Wiring

Ethernet Wiring

Home Router



Power Wiring

Anker Charger

USB-C #1 —> pi-node-1

USB-C #2 —> pi-node-2

USB-C #3 —> pi-node-3

pi-node-1 USB 3.0 —> 1TB SSD

Fan 1

Red —> pi-node-1 GPIO Pin 4

Black —> pi-node-1 GPIO Pin 6

Fan 2

Red —> pi-node-3 GPIO Pin 4

Black —> pi-node-3 GPIO Pin 6

Ethernet Cable Construction

The SAIL environment uses custom Ethernet cables to connect the Raspberry Pi cluster nodes, network switch, and administrative workstation. Building custom cables allows cable lengths to be matched to the physical layout of the environment while providing hands-on experience with basic network infrastructure.

Materials

- Cat5e or Cat6 Ethernet cable
- RJ45 connectors
- RJ45 crimping tool
- Cable stripper
- Cable tester

Step 1: Measure Cable Length

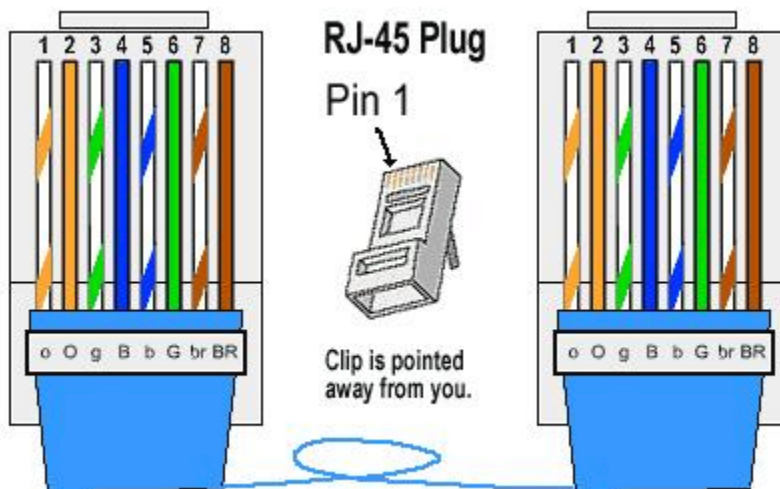
1. Measure the distance between the devices to be connected.
2. Add approximately 12 inches (30 cm) of extra cable to allow for routing and future adjustments.
3. Cut the cable to the desired length.

Step 2: Strip the Cable Jacket

1. Using the cable stripper, remove approximately 1 inch (2.5 cm) of the outer cable jacket from the end of the cable.
2. Take care to avoid damaging the internal conductors.

Step 3: Arrange Conductors

1. Untwist the wire pairs and arrange them according to the T568B standard:
 - a. White/Orange
 - b. Orange
 - c. White/Green
 - d. Blue
 - e. White/Blue
 - f. Green
 - g. White/Brown
 - h. Brown



2. Flatten the conductors and place them in the correct order.

Step 4: Trim Conductors

1. Hold the conductors firmly and trim them to an equal length approximately 0.5 inches (1.3 cm) from the cable jacket.

Step 5: Insert RJ45 Connector

1. Insert the conductors into the RJ45 connector while maintaining the T568B wire order.

2. Verify that all conductors reach the end of the connector and remain in the correct sequence.
3. Verify that the cable jacket extends into the connector body.

Step 6: Crimp Connector

1. Insert the connector into the crimping tool.
2. Fully compress the crimping tool to secure the conductors and strain relief.

Step 7: Repeat for Opposite End

1. Repeat Steps 2 through 6 using the same T568B wiring order.

Step 8: Test Cable

1. Connect both ends of the cable to a cable tester.
2. Verify continuity on all eight conductors.
3. Replace and recrimp any connector that fails testing.

Verification

1. Connect the completed cable between two network devices.
2. Verify link lights are present on both devices and confirm network connectivity.

Physical Environment Assembly

Materials

- Raspberry Pi 4 (8GB) x3
- Raspberry Pi tower or cluster case
- MicroSD cards
- 1TB external SSD
- Network switch
- Ethernet cable
- RJ45 connectors
- RJ45 crimping tool
- Cable tester
- Raspberry Pi power supplies

Step 1: Prepare the MicroSD Cards

1. Verify that the prepared and labeled MicroSD cards are available for installation.
2. Configure hostname, SSH, username/password, locale, etc.
3. Label each card (**pi-node-1**, **pi-node-2**, **pi-node-3**).

Step 2: Install Storage Media

1. Insert the prepared MicroSD cards into the Raspberry Pi systems.
2. Connect the 1TB external SSD to the node designated as pi-node-1.
3. Verify that all storage devices are seated securely.

Step 3: Assemble the Raspberry Pi Tower

1. Assemble the Raspberry Pi tower according to the Hardware Layout and Hardware Wiring diagrams in this section. Refer to the manufacturer's instructions only for the mechanical assembly of the tower.
2. Install all Raspberry Pi systems into the tower.

3. Install any supplied heatsinks, cooling fans, or mounting hardware.
4. Verify that all components are mounted securely and that cooling components operate without obstruction.

Step 4: Construct Ethernet Cables

1. Measure the required cable lengths based on the planned location of the tower and network switch.
2. Add approximately 12 inches (30 cm) to each measurement to allow for cable routing and future adjustments.
3. Construct and test Ethernet cables using the Ethernet Cable Construction procedure documented earlier in this section.
4. Label cables as desired.

Step 5: Install Network Infrastructure

1. Place the Raspberry Pi tower and network switch in their intended locations.
2. Connect all Raspberry Pi systems to the network switch.
3. Connect the network switch to the local network.
4. Verify that all network connections are fully seated.

Step 6: Connect Power

1. Connect power supplies to the Raspberry Pi systems and network switch.
2. Route cables to minimize strain on connectors and maintain airflow around equipment.

Step 7: Power On the Environment

1. Apply power to the network switch and Raspberry Pi systems.
2. Observe power indicators, network link lights, and cooling operation.
3. Allow the systems to complete their initial startup sequence.

Step 8: Verify Physical Infrastructure

Verify:

- Raspberry Pi tower is assembled correctly
- Storage devices are connected
- Ethernet cables pass testing
- Network link lights are active
- Cooling components are operating
- All systems receive power

Completion Criteria

Physical environment assembly is complete when:

- The Raspberry Pi tower is assembled
- Storage devices are installed
- Network cabling is connected and tested
- Network connectivity is established
- All systems power on successfully
- The environment is ready for operating system installation

Section 5: Network Configuration

Network Planning and IP Address Allocation

Before configuring persistent IP addresses, the local network configuration was reviewed to determine how the cluster nodes would be addressed and managed. Establishing an addressing plan early helps avoid conflicts with existing devices and simplifies SSH access, and cluster administration.

The first step was identifying the address currently assigned to the system. A quick review of assigned addresses was performed using:

```
ifconfig | grep "inet"
```

Because multiple addresses were returned, the active network adapter was identified using:

```
networksetup -listallhardwareports
```

After identifying the active network interface, its assigned address was obtained using:

```
ipconfig getifaddr en0
```

The resulting address was used to determine the address range used by the local network. Readers following this process should note the first three portions of the address, which typically indicate the local network range.

Next, the default gateway was identified using:

```
netstat -nr | grep default
```

```
(base) user@MacBook-Pro ~ % netstat -nr | grep default
default      10.0.0.1      UGScg          en0
default      fe80::7a23:aeff:fe7b:434f%en0  UGcg          en0
default      fe80::%utun0  UGcIg         utun0
default      fe80::%utun1  UGcIg         utun1
default      fe80::%utun2  UGcIg         utun2
default      fe80::%utun3  UGcIg         utun3
(base) user@MacBook-Pro ~ %
```

The gateway address was entered into a web browser to access the router or gateway administration interface.

Within the administration interface, the local network settings were reviewed to identify:

- The gateway address
- The subnet mask
- The DHCP address range
- Available IP reservation features

The review confirmed that the gateway supported DHCP reservations. DHCP reservations allow specific devices to consistently receive the same IP address while still using DHCP for network configuration. This approach provides the convenience of DHCP while maintaining predictable addresses for cluster infrastructure. Based on the available address space, three addresses were selected and reserved for the Kubernetes nodes. Assigning predictable addresses simplifies administration, automation, and troubleshooting.

Planned DHCP Reservations

To provide predictable node addresses for SSH access, cluster administration, Ansible automation, and troubleshooting, DHCP reservations is configured on the local gateway.

System	Role	Reserved IP
pi-node-1	k3s control plane, worker, SSD host	10.0.0.20
pi-node-2	k3s worker	10.0.0.21
pi-node-3	k3s worker	10.0.0.22

The reservations can be created through the gateway administration interface after the Raspberry Pi systems have been connected to the network and obtained initial DHCP leases.

The reservation process consists of:

1. Log in to the gateway administration interface.
2. Navigate to Connected Devices.
3. Select Add Device With Reserved IP.
4. Identify the Raspberry Pi by hostname, MAC address, or current DHCP address and select it from the device list.
5. Assign the planned IP address.
6. Save the configuration.
7. Repeat for the remaining nodes.
8. Reboot the Raspberry Pi systems or renew their DHCP leases.

Once completed, the cluster nodes consistently receives the same addresses whenever they reconnect to the network.

If there is no direct access to the gateway, have the nodes request the addresses. Check what the automatically assigned addresses are with:

Node	Command	Result IP
pi-node-1	<code>ping pi-node-1.local</code>	10.0.0.155
pi-node-2	<code>ping pi-node-2.local</code>	10.0.0.119
pi-node-3	<code>ping pi-node-3.local</code>	10.0.0.149

To change them to the planned static IPs, start with node 1:

1. SSH into the node: `ssh sailadmin@pi-node-1.local`
2. ID the ethernet connection: `nmcli connection show`
 - a. For all nodes, that connection is `Wired connection 1` with device `eth0`
3. Configure the static IP:

```
sudo nmcli connection modify "Wired connection 1" \  
    ipv4.method manual \  
    ipv4.addresses 10.0.0.20/24 \  
    ipv4.gateway 10.0.0.1 \  
    ipv4.dns "10.0.0.1"
```
4. Verify the saved configuration before activating it:

```
nmcli -f ipv4.method,ipv4.addresses,ipv4.gateway,ipv4.dns \  
connection show "Wired connection 1"
```

Expected result for node 1:

```
ipv4.method:      manual  
ipv4.addresses:   10.0.0.20/24  
ipv4.gateway:     10.0.0.1  
ipv4.dns:         10.0.0.1
```
5. Activate the new config: `sudo nmcli connection up "Wired connection 1"`
6. Verify the node's new address: `ip -4 addr show eth0`

```
inet 10.0.0.20/24  
valid_lft forever preferred_lft forever
```
7. Repeat these steps with nodes 2 & 3
8. Exit ssh and verify all three from the computer:

```
ping -c 2 10.0.0.20  
ping -c 2 10.0.0.21  
ping -c 2 10.0.0.22
```

IP Address Verification

After configuring the static IP addresses, verify that each node is reachable at its assigned address using the ping commands above. All three nodes should respond from their planned addresses: `10.0.0.20`, `10.0.0.21`, and `10.0.0.22`.

Environment Shutdown Procedure

The SAIL environment should be shut down cleanly before disconnecting power. This allows the Raspberry Pi systems to complete disk operations and shut down the operating system before power is removed.

Step 1: Shut Down pi-node-1

1. Open a terminal on the computer.
2. Connect to **pi-node-1**: `ssh sailadmin@10.0.0.20`
3. Shut down the node: `sudo shutdown -h now`
4. Wait for the SSH connection to close.

Step 2: Shut Down pi-node-2

1. Connect to **pi-node-2**: `ssh sailadmin@10.0.0.21`
2. Shut down the node: `sudo shutdown -h now`
3. Wait for the SSH connection to close.

Step 3: Shut Down pi-node-3

1. Connect to **pi-node-3**: `ssh sailadmin@10.0.0.22`
2. Shut down the node: `sudo shutdown -h now`
3. Wait for the SSH connection to close.

Step 4: Disconnect Tower Power

1. Wait approximately 30 seconds after shutting down the final Raspberry Pi.
2. Unplug the tower from its power source.
3. Verify that the Raspberry Pi systems, cooling fans, and network switch are no longer receiving power.

Section 6: Kubernetes setup

The SAIL environment uses k3s to provide Kubernetes orchestration across the Raspberry Pi cluster. k3s was selected because it is a lightweight Kubernetes distribution designed for environments with limited computing resources while maintaining compatibility with the standard Kubernetes APIs and tooling. This allows SAIL to provide a realistic Kubernetes environment using Raspberry Pi hardware without the overhead of a full Kubernetes installation.

The cluster consists of one control plane node and two worker nodes. The control plane is hosted on pi-node-1, which also participates as a schedulable worker node. The remaining Raspberry Pi systems function as worker nodes that host workloads distributed by Kubernetes. This configuration provides a balance between efficient hardware utilization and a realistic multi-node cluster suitable for infrastructure management, monitoring, security testing, and AI-assisted operational analysis.

Kubernetes Installation

The following procedures install k3s on the Raspberry Pi cluster and create the Kubernetes environment.

Step 1: Verify Network Connectivity

1. Open a terminal on the computer.
2. Verify that each Raspberry Pi is accessible using SSH: `ssh <username>@10.0.0.20`
3. After successfully connecting, run: `exit`
4. Repeat for: `ssh <username>@10.0.0.21` and `ssh <username>@10.0.0.22`
5. Resolve any connectivity issues before continuing.

Step 2: Install the k3s Control Plane

1. Open a terminal on the computer.
2. Connect to **pi-node-1**: `ssh <username>@10.0.0.20`
3. Run the following command: `curl -sL https://get.k3s.io | sh -`
4. Wait for the installation to complete before proceeding.

Step 3: Verify the Control Plane Installation

1. While still connected to **pi-node-1**, run: `sudo kubectl get nodes`
2. Verify that **pi-node-1** reports a status of **Ready**.

Step 4: Retrieve the Cluster Join Token

1. While still connected to **pi-node-1**, display the node join token.
 - a. `sudo cat /var/lib/rancher/k3s/server/node-token`
2. Copy the entire token.
3. The token is required when joining the worker nodes to the cluster.

Step 5: Install k3s on pi-node-2

1. Disconnect from **pi-node-1**: `exit`
2. Connect to **pi-node-2**: `ssh <username>@10.0.0.21`
3. Run: `curl -sL https://get.k3s.io | K3S_URL=https://10.0.0.20:6443 K3S_TOKEN=<node-token> sh -`
4. Replace **<node-token>** with the token copied in Step 4.
5. Wait for the installation to complete.

Step 6: Install k3s on pi-node-3

1. Disconnect from **pi-node-2**: `exit`
2. Connect to **pi-node-3**: `ssh <username>@10.0.0.22`
3. Run: `curl -sL https://get.k3s.io | K3S_URL=https://10.0.0.20:6443 K3S_TOKEN=<node-token> sh -`
4. Replace **<node-token>** with the token copied in Step 4.
5. Wait for the installation to complete.

Step 7: Verify Cluster Membership

1. Disconnect from **pi-node-3**: `exit`
2. Reconnect to **pi-node-1**: `ssh <username>@10.0.0.20`
3. Run: `sudo kubectl get nodes`
4. Verify all three nodes report a status of **Ready**.

Example output:

NAME	STATUS	ROLES	AGE
pi-node-1	Ready	control-plane,master	
pi-node-2	Ready	<none>	
pi-node-3	Ready	<none>	

Step 8: Verify Cluster Information

1. Run: `sudo kubectl cluster-info`
2. Verify that the Kubernetes API server is accessible.

Step 9: Verify Kubernetes System Pods

1. Run: `sudo kubectl get pods --all-namespaces`
2. Verify that all Kubernetes system pods report a status of **Running** or **Completed**.

Step 10: Deploy a Test Workload

1. Create a test deployment: `sudo kubectl create deployment nginx --image=nginx`
2. Verify the deployment: `sudo kubectl get deployments`
3. Verify the pod: `sudo kubectl get pods -o wide`

4. Confirm that the pod reaches the **Running** state.

Step 11: Create a Test Service

1. Run: `sudo kubectl expose deployment nginx --port=80 --type=ClusterIP`
2. Verify the service: `sudo kubectl get services`

Step 12: Remove the Test Workload

1. Run: `sudo kubectl delete service nginx`
`sudo kubectl delete deployment nginx`
2. Verify that the deployment and service have been removed.

Completion Criteria

The Kubernetes installation is complete when:

- k3s is installed on all three Raspberry Pi systems.
- All three nodes report a status of **Ready**.
- Kubernetes system pods are healthy.
- A test workload deploys successfully.
- A Kubernetes service is created successfully.
- The test deployment and service are removed successfully.
- Baseline cluster information has been recorded.

Section 7: Workload Design

SAIL operates multiple workloads to create an active environment for infrastructure, security, automation, and AI-assisted operations. These workloads generate ongoing activity that can be monitored and analyzed while also providing opportunities to test failures, security events, investigation, and recovery.

The environment includes four primary workloads:

- **Application workload:** Containerized applications generate API requests, authentication activity, data processing, storage operations, errors, and other application behavior.
- **Security workload:** Security tools assess systems and assets for vulnerabilities and security weaknesses while monitoring the running environment for security-relevant behavior. Security activity produces findings and detections that can be investigated using evidence collected throughout SAIL.
- **Automated/AI workload:** Automated processes and AI agents interact with applications and services while the environment is running. These processes create sustained activity, generate different usage and security patterns, and support investigation and remediation workflows.
- **Operational workload:** Kubernetes, containers, hosts, storage, and networking generate activity through deployments, scheduled processes, configuration changes, resource usage, failures, troubleshooting, and recovery operations.

These workloads create the data used throughout SAIL. Monitoring and security systems collect logs, metrics, traces, vulnerability findings, runtime detections, application records, and system information that can be used during troubleshooting and security investigations.

Application Workload

The application workload provides functioning services for SAIL to host and operate. Applications run as containerized workloads on the Kubernetes cluster and provide real targets for deployment, monitoring, security assessment, automated activity, troubleshooting, and investigation.

The application workload includes:

1. **Interactive application activity:** Applications expose APIs or other interfaces that can receive requests and perform useful processing. Requests can involve authentication, authorization, data access, application logic, and persistent storage.
2. **Persistent application state:** Application data remains available across container and pod restarts. Database activity also provides another service dependency that can be monitored, secured, disrupted, and recovered.
3. **Security-relevant application behavior:** Authentication attempts, access-control decisions, input validation, privileged operations, configuration changes, errors, and other significant actions produce records that can be correlated with infrastructure and security telemetry.
4. **External asset interaction:** Application components can interact with authorized external assets when a workload requires information outside the SAIL cluster. This allows security-oriented services to perform useful work against assets such as owned websites or repositories rather than operating only against simulated targets.

The exact functionality of the initial application is determined after the complete SAIL workload is defined. Its design needs to provide useful application behavior while also supplying enough activity and security context for the other SAIL workloads.

Security Workload

The security workload provides SAIL with active security assessment, detection, and investigation capabilities.

1. **Assessment:** SAIL examines systems and assets for vulnerabilities and security weaknesses. Assessments can cover SAIL infrastructure and applications along with external assets owned or administered by the operator. Assessments can identify vulnerable software, exposed services, insecure configurations, application weaknesses, and other security findings. Results are recorded so findings can be reviewed, investigated, remediated, and compared with later assessments.
2. **Detection:** SAIL monitors running systems and applications for security-relevant behavior. This includes authentication failures, unauthorized access attempts, unexpected processes, container activity, configuration changes, suspicious requests, unusual application activity, and other behaviors identified by the security tooling.
3. **Investigation:** Security findings and detections are supported with evidence collected across the environment. Investigations can use application activity, system state, vulnerability results, logs, metrics, traces, runtime security information, and related records to determine what occurred and assess its significance. AI-assisted investigation capabilities can use this evidence to correlate activity and support analysis.

4. **Verification:** Security assessments can be repeated after remediation to determine whether an identified weakness has been corrected. Detection and monitoring data can also confirm whether affected systems have returned to their expected state.

The security workflow follows:

Assess → Detect → Collect Evidence → Investigate → Remediate → Verify

This workflow provides a common process for handling security findings and events throughout SAIL. It connects the security workload with the application, automated/AI, and operational workloads so activity occurring in one part of the environment can be examined using evidence produced elsewhere in the system.

Automated/AI Workload

The automated/AI workload provides sustained activity and AI-assisted operational capabilities across SAIL. Automation performs repeatable work while agents make bounded decisions based on the state of the environment and the information available to them.

The workload includes three agent roles:

1. **Activity Agent:** Generates sustained interaction with applications and services. The agent can represent different patterns of normal and security-relevant behavior, vary the frequency and sequence of its actions, and create enough activity for monitoring systems to establish patterns and detect changes.
2. **Investigation Agent:** Examines security findings, alerts, and operational problems by retrieving relevant evidence from SAIL. The agent can correlate application records, logs, metrics, traces, runtime detections, vulnerability results, and current system state to develop an evidence-supported assessment.
3. **Remediation Agent:** Uses investigation results to prepare corrective actions. Remediation activity is recorded and followed by verification so the outcome can be compared with the original finding or incident.

Agent activity is also observable. Requests, tool calls, decisions, investigation inputs, outputs, and remediation actions generate records that can be reviewed alongside the systems the agents operate against.

Operational Workload

The operational workload represents the normal administration and behavior of the SAIL infrastructure. It provides the system-level activity required to operate applications and security services across the physical hosts and Kubernetes cluster.

The workload includes:

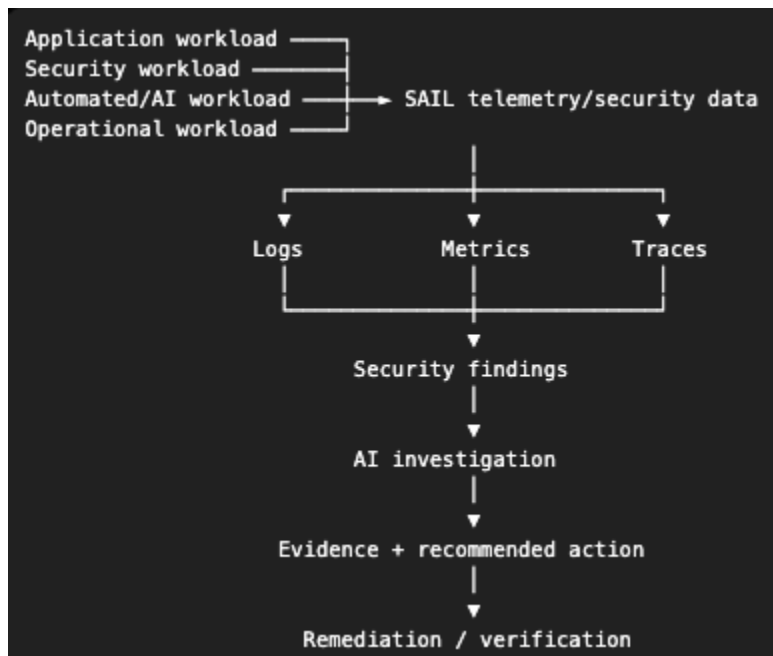
1. **Cluster operations:** Kubernetes schedules and manages application, security, monitoring, automation, and AI workloads across the available nodes. Deployments, pod creation, restarts, scheduling decisions, and service changes generate operational activity.
2. **System operations:** Hosts generate CPU, memory, storage, network, process, and service activity while SAIL is running. Changes in these resources provide infrastructure-level context for application and security events.

3. **Deployment and configuration activity:** Application releases, container images, infrastructure configuration, Kubernetes resources, and automated configuration changes create a record of how the environment changes over time.
4. **Failure and recovery activity:** Service outages, pod failures, resource pressure, storage problems, network problems, and configuration errors provide scenarios for detection and troubleshooting. Recovery activity provides evidence that corrective actions restored the affected system.

Operational data provides context for determining whether an application or security event is isolated or connected to a broader change in the environment.

Integrated Workload Flow

The four workloads operate as parts of the same environment rather than as separate demonstrations. Application and operational workloads produce normal system activity. Security systems assess assets and monitor the environment. Automated processes and AI agents interact with these systems and generate additional activity.



Section 8: Application Development and Deployment

Application Design

Triage is a social media application that provides a realistic workload for SAIL while supporting continued development as an independent application. Users can create accounts, authenticate, maintain profiles, publish posts, comment and react to posts, follow other users, and view a chronological feed. User and administrative permissions control access to

application functions and data. Triage exposes its functionality through a REST API and stores persistent data in PostgreSQL.

Automated users can generate substantial activity by exercising authentication, content creation, feed retrieval, social relationships, authorization checks, database operations, and concurrent API requests. The volume and frequency of these interactions can be varied to create different workload levels for monitoring application performance and infrastructure behavior. Triage also generates security-relevant activity through authentication attempts, permission checks, administrative operations, validation failures, user actions, and application errors.

The SAIL implementation focuses on the functionality needed to provide a useful application workload without requiring the complete feature set of a production social media platform. Application activity can be correlated with monitoring and security data collected elsewhere in SAIL, while the initial architecture provides a foundation for continued development of Triage after SAIL is complete.

The SAIL implementation of Triage includes:

- User registration and authentication.
- User profiles and account management.
- User and administrative roles and permissions.
- Creation, editing, deletion, and retrieval of posts.
- Comments and reactions.
- Follow and unfollow relationships.
- A chronological user feed.
- REST API access and request validation.
- Persistent application data.
- Application and security-relevant logging.
- Health and application status endpoints.
- Support for sustained automated user activity.

The application uses:

Technology	Purpose
Python	Programming language
FastAPI	Web/API framework
Pydantic	Request/response validation and schemas
Uvicorn	Runs the FastAPI application
PostgreSQL	Application database
SQLAlchemy	ORM/database access
psycopg	Connects the application to PostgreSQL
Alembic	Database schema migrations

PyJWT	Authentication tokens
pwdlib / Argon2	Password hashing
pytest	Automated testing
Black	Code formatting
Ruff	Linting/code quality
dotenv	Loads application configuration from .env files into env variables.

Application Structure

The `api` package handles REST endpoints, `models` contains database models, `services` contains application logic that does not belong directly in an endpoint, `schemas` handles request and response validation, and `tests` contains automated tests for the application.

```
sail-triage/
├── app/
│   ├── __init__.py
│   ├── config.py
│   ├── extensions.py
│   └── api/
│       ├── auth.py
│       ├── users.py
│       ├── posts.py
│       ├── comments.py
│       ├── reactions.py
│       └── follows.py
│   ├── models/
│       ├── user.py
│       ├── post.py
│       ├── comment.py
│       ├── reaction.py
│       └── follow.py
│   ├── services/
│       ├── auth.py
│       └── feed.py
│   └── schemas/
│       ├── auth.py
│       ├── user.py
│       ├── post.py
│       ├── comment.py
│       └── reaction.py
├── tests/
│   ├── test_auth.py
│   ├── test_users.py
│   ├── test_posts.py
│   ├── test_comments.py
│   ├── test_reactions.py
│   ├── test_follows.py
│   └── test_feed.py
├── .sail-triage-env/
├── .env
├── .env.example
├── requirements.txt
├── run.py
└── README.md
```

Development follows standard Python practices, including PEP 8 conventions, type hints where appropriate, automated formatting and linting, separation of application responsibilities, and automated testing.

Application Behavior

At a high level, Triage supports the following application workflow:

1. **Register and authenticate users** using secure authentication tokens.
2. **Manage accounts and profiles** with user and administrative permissions.
3. **Create and manage posts** including publishing, retrieving, editing, and deleting.
4. **Interact with content** through comments and reactions.
5. **Manage social relationships** through following and unfollowing users.
6. **Generate a chronological feed** based on followed users and content.
7. **Validate requests and enforce authorization** before accessing protected functions or data.
8. **Store and retrieve application data** through PostgreSQL.
9. **Record security-relevant activity** including authentication, permission checks, administrative actions, validation failures, and errors.
10. **Report application health** for monitoring Triage and its dependencies.
11. **Support sustained automated activity** at varying workload levels.

Application Deployment

The application completes in the following order:

1. [Create the local project and Python virtual environment.](#) **Complete**
2. [Configure VS Code to use the project environment.](#) **Complete**
3. [Configure Git for the application.](#) **Complete**
4. [Install the initial application and development dependencies.](#) **Complete**
5. [Make the application project structure for Triage.](#) **Complete**
6. [Configure application settings and environment variables.](#) **Complete**
7. [Configure PostgreSQL for local development.](#) **Complete**
8. [Configure SQLAlchemy and database connectivity.](#) **Complete**
9. [Configure Alembic database migrations.](#) **Complete**
10. [Implement the user data model and database schema.](#) **Complete**
11. Implement user registration, authentication, and password security.
12. Implement user profiles, roles, and permissions.
13. Implement posts and associated API endpoints.
14. Implement comments and reactions.
15. Implement follow and unfollow relationships.
16. Implement the chronological feed.
17. Implement Pydantic request and response schemas.
18. Add application and security-relevant logging.
19. Add health endpoints and application error handling.

20. Create automated tests with pytest.
21. Run Black and Ruff against the application.
22. Run and test the complete application locally.
23. Verify FastAPI validation and OpenAPI documentation.
24. Generate sustained automated requests to verify Triage can operate as a variable workload.
25. Containerize the completed application.
26. Deploy Triage and PostgreSQL to the k3s cluster.
27. Verify that the deployed application is reachable, persistent, and functioning correctly.

Create the Application Environment

Step 1: Create the application folder and virtual environment

Inside the existing **SAIL** repository, create the application folder:

```
SAIL/  
├── README.md  
├── docs/  
└── sail-triage/
```

- a. In Terminal, from the root of the SAIL repo:

```
mkdir -p sail-triage  
cd sail-triage
```

- b. Create the virtual environment:

```
python3 -m venv .sail-triage-env
```

- c. Activate it:

```
source .sail-triage-env/bin/activate
```

- d. Verify it:

```
which python  
python --version
```

The **which python** result should point to something like:

```
.../SAIL/sail-triage/.sail-triage-env/bin/python
```

At this point, the only thing created automatically inside **sail-triage** is:

```
sail-triage/  
└── .sail-triage-env/
```

Step 2: Configure VS Code to Use the Environment

- a. Press **Cmd + Shift + P**.

- b. Search for:

Python: Select Interpreter

- c. Select: Enter interpreter path → Find.

- d. Select:

sail-triage/.sail-triage-env/bin/python

Step 3: Configure Git for the Application

Since **SAIL** is already a Git repository, do not run **git init** again.

- a. From inside **sail-triage**, confirm you are inside the existing repository:

```
git status
```

b. Add the virtual environment to the repository's `.gitignore`.

From the SAIL repository root, open or create `.gitignore` and add:

```
sail-triage/.sail-triage-env/
```

c. Verify Git is ignoring it:

```
git status
```

d. Create a development branch for the application work:

```
git switch -c develop
```

e. Verify the branch:

```
git branch
```

You should see:

```
* develop
```

```
main
```

f. in a `.gitignore` at the file base level:

```
# Python virtual environment
sail-triage/.sail-triage-env/
```

```
# Python generated files
```

```
__pycache__/
```

```
*.py[co]
```

```
# Testing and tooling caches
```

```
.pytest_cache/
```

```
.ruff_cache/
```

```
.coverage
```

```
htmlcov/
```

```
# Environment variables and secrets
```

```
.env
```

```
.env.*
```

```
!.env.example
```

```
# macOS
```

```
.DS_Store
```

```
# VS Code local settings
```

```
.vscode/
```

```
# Logs
```

```
*.log
```

g. Make an initial commit for the application folder/setup once the structure you want tracked exists. For now, if `sail-triage` contains only the ignored virtual environment, there is nothing useful to commit yet.

Step 4: Install Application and Development Dependencies

With `.sail-triage-env` activated and the terminal inside `sail-triage`:

a. Create the requirements file:

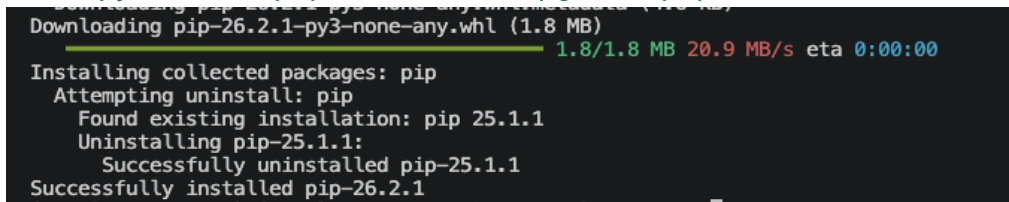
```
touch requirements.txt
```

b. Open `requirements.txt` in VS Code and add:

```
Flask
Flask-RESTX
pytest
black
ruff
```

c. Upgrade pip:

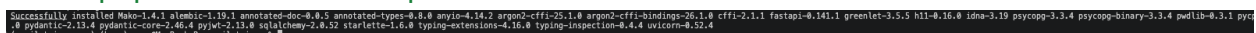
```
python -m pip install --upgrade pip
```

A terminal window showing the process of upgrading pip. It starts with downloading pip-26.2.1-py3-none-any.whl (1.8 MB) at 20.9 MB/s. Then it shows the installation of collected packages, attempting to uninstall the existing pip-25.1.1, and finally successfully installing pip-26.2.1.

```
Downloading pip-26.2.1-py3-none-any.whl (1.8 MB)
1.8/1.8 MB 20.9 MB/s eta 0:00:00
Installing collected packages: pip
Attempting uninstall: pip
Found existing installation: pip 25.1.1
Uninstalling pip-25.1.1:
Successfully uninstalled pip-25.1.1
Successfully installed pip-26.2.1
```

d. Install the dependencies from the requirements file:

```
pip install -r requirements.txt
```

A terminal window showing the successful installation of dependencies from requirements.txt. It lists various packages like Make, alembic, annotated-doc, annotated-types, anyio, argon2, fastapi, greenlet, h11, idna, pycopp, pycopp-binary, pyc, pydantic, pydantic-core, pyjwt, sqlalchemy, starlette, typing-extensions, typing-inspection, uvicorn, etc.

```
Successfully installed Make-1.4.1 alembic-1.19.1 annotated-doc-0.0.5 annotated-types-0.8.0 anyio-4.14.2 argon2-cffi-25.1.0 argon2-cffi-bindings-26.1.0 cffi-2.1.1 fastapi-0.141.1 greenlet-3.5.5 h11-0.16.0 idna-3.19 pycopp-3.3.4 pycopp-binary-3.3.4 pyc-0.0 pydantic-2.13.4 pydantic-core-2.46.4 pyjwt-2.13.0 sqlalchemy-2.0.52 starlette-1.6.0 typing-extensions-4.16.0 typing-inspection-0.4.4 uvicorn-0.52.4
```

e. Verify the installed packages:

```
pip list
```

Confirm that the list includes:

```
fastapi
pydantic
uvicorn
sqlalchemy[asyncio]
pycopp[binary]
alembic
pyjwt
pewlib[argon2]
pytest
black
ruff
```

Once verified, put the version numbers in the requirements.txt.

f. Verify that Git now sees the application directory and requirements file:

```
git status
```

You should see `sail-triage/requirements.txt` as an untracked file. The `.sail-triage-env` directory should **not** appear because it is ignored.

Step 5: Create the Application Project Structure

a. From inside `sail-triage`, create the directories:

```
mkdir -p app/api app/models app/services app/schemas tests
```

b. Create the core application files:

```
touch app/__init__.py \
```

```
app/config.py \
app/extensions.py \
run.py
```

c. Create the application README:

```
touch README.md
```

d. Create the Triage API files:

```
touch app/api/auth.py \
app/api/users.py \
app/api/posts.py \
app/api/comments.py \
app/api/reactions.py \
app/api/follows.py
```

e. Create the database model files:

```
touch app/models/user.py \
app/models/post.py \
app/models/comment.py \
app/models/reaction.py \
app/models/follow.py
```

f. Create the service files:

```
touch app/services/auth.py \
app/services/feed.py
```

g. Create the Pydantic schema files:

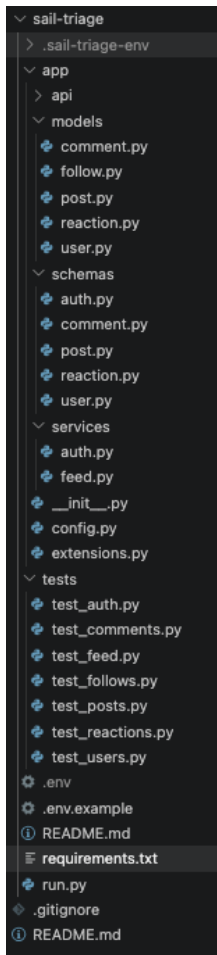
```
touch app/schemas/auth.py \
app/schemas/user.py \
app/schemas/post.py \
app/schemas/comment.py \
app/schemas/reaction.py
```

h. Create the test files:

```
touch tests/test_auth.py \
tests/test_users.py \
tests/test_posts.py \
tests/test_comments.py \
tests/test_reactions.py \
tests/test_follows.py \
tests/test_feed.py
```

i. Verify the structure: `find app tests -maxdepth 2 -type f | sort`

Your structure should now be:



Step 6: Configure Application Settings and Environment Variables

a. Create an environment file for local development:

```
touch .env
```

b. Create a safe template that can be committed to Git:

```
touch .env.example
```

c. In `.env`, add the local application settings we know we will need:

```
FLASK_ENV=development
```

```
FLASK_DEBUG=1
```

d. Put the same variable names in `.env.example`, but without any sensitive values:

```
FLASK_ENV=development
```

```
FLASK_DEBUG=1
```

e. Confirm the root `.gitignore` includes:

```
.env
```

```
.env.*
```

```
!.env.example
```

f. In `app/config.py`, we will later load configuration from environment variables rather than hardcoding it into the application.

Step 7: Configure PostgreSQL for Development

a. First, check whether PostgreSQL is already installed on your Mac:

```
psql --version
```

b. If `psql` is not found, install PostgreSQL with Homebrew:

```
brew install postgresql@17
```

```
Success. You can now start the database server using:
```

```
'/opt/homebrew/Cellar/postgresql@17/17.11/bin/pg_ctl' -D '/opt/homebrew/var/postgresql@17' -l logfile start
```

c. Start PostgreSQL:

```
brew services start postgresql@17
```

```
(.sail-triage-env) (base) user@MacBook-Pro sail-triage % brew services start postgresql@17  
=> Successfully started `postgresql@17` (label: homebrew.mxcl.postgresql@17)
```

d. Verify the server is running:

```
pg_isready
```

Response should be similar to:

```
/var/run/postgresql:5432 - accepting connections
```

```
(.sail-triage-env) (base) user@MacBook-Pro sail-triage % pg_isready  
/tmp:5432 - accepting connections
```

e. Create a PostgreSQL database for Triage:

```
createdb triage
```

f. Verify that it exists:

```
psql -l
```

You should see `triage` in the database list.

```
(.sail-triage-env) (base) user@MacBook-Pro sail-triage % psql -l
```

Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
postgres	user	UTF8	libc	en_US.UTF-8	en_US.UTF-8			
template0	user	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/user +
template1	user	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/user +
triage	user	UTF8	libc	en_US.UTF-8	en_US.UTF-8			=c/user +

(4 rows)

g. Open the PostgreSQL prompt:

```
psql postgres
```

Should get a prompt similar to:

```
postgres=#
```

h. Create a Triage database user. Replace `your_password_here` with a chosen password:

```
CREATE USER triage_app WITH PASSWORD 'your_password_here';
```

i. Make `triage_app` the owner of the `triage` database:

```
ALTER DATABASE triage OWNER TO triage_app;
```

j. Connect to the `triage` database:

```
\c triage
```

k. Make `triage_app` the owner of the `public` schema (Alembic to create/modify tables later):

```
ALTER SCHEMA public OWNER TO triage_app;
```

l. Exit PostgreSQL:

```
\q
```

m. Test the new application account:

```
psql -U triage_app -d triage -h localhost
```

It should ask for the password you just created. After entering it, should reach:

```
trriage=>
```

Then run:

```
SELECT current_user, current_database();
```

```
trriage=> SELECT current_user, current_database();
current_user | current_database
-----+-----
trriage_app  | triage
(1 row)
```

n. Now exit PostgreSQL:

```
\q
```

o. Then update DATABASE_URL in your .env:

```
DATABASE_URL=postgresql+psycopg://trriage_app:trriage_et@localhost:5432/trriage
```

Keep the password only in .env; .env.example should continue to have:

```
DATABASE_URL=
```

p. To shut down the database:

```
brew services stop postgresql@17
```

Verify it stopped:

```
pg_isready
```

Should get something like:

```
localhost:5432 - no response
```

Step 8: Configure SQLAlchemy and database connectivity.

a. Open:

```
app/extensions.py
```

Add:

```
"""Application extensions for Triage."""
```

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
```

```
from app.config import Config
```

```
engine = create_engine(
    Config.DATABASE_URL,
    pool_pre_ping=True,
)
```

```
SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    autocommit=False,
)
```

b. Create a small database test file:

```
touch app/database_test.py
```

c. Add:

```
"""Verify Triage can connect to the configured database."""

from sqlalchemy import text

from app.extensions import engine

def test_database_connection():
    """Connect to PostgreSQL and return the current database
    user."""

    with engine.connect() as connection:
        result = connection.execute(
            text("SELECT current_user, current_database();")
        )

        return result.fetchone()

if __name__ == "__main__":
    print(test_database_connection())
```

d. Run it from inside `sail-triage` with your virtual environment active:

```
python -m app.database_test
```

Expected output should look similar to:

```
('triage_app', 'triage')
```

e. Move the file to tests and rename to a better name for explanation

```
mv app/database_test.py tests/test_database_connection.py
```

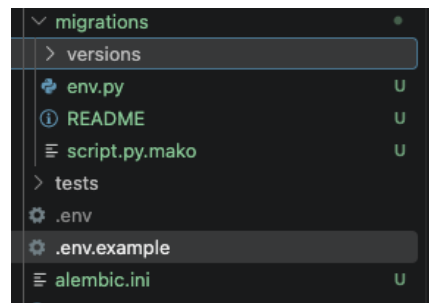
Step 9: Configure Alembic database migrations

a. From inside `sail-triage`, initialize Alembic:

```
alembic init migrations
```

This creates:

```
alembic.ini
migrations/
├── env.py
├── script.py.mako
└── versions/
```



b. Open:

`alembic.ini`

Find:

`sqlalchemy.url =` -> Make it blank.

Want Alembic to use the same `DATABASE_URL` from `.env`

c. Open:

`migrations/env.py`

Near the top, add:

`from app.config import Config`

Then find:

`config = context.config`

Immediately after it, add:

`config.set_main_option("sqlalchemy.url",
Config.DATABASE_URL)`

d. For now, leave:

`target_metadata = None`

Haven't yet created the SQLAlchemy models. In the next step, replace this with the app's model metadata so Alembic can detect schema changes automatically.

e. Verify Alembic can connect to PostgreSQL:

`alembic current`

```
• (.sail-triage-env) (base) user@MacBook-Pro sail-triage % alembic current
INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO [alembic.runtime.migration] Will assume transactional DDL.
```

f. Verify Alembic itself is configured:

`alembic heads`

Alembic is initialized, is reading the Triage database configuration, and can connect to PostgreSQL successfully.

Step 10: Implement the user data model and database schema

Define the SQLAlchemy base, implement this model in `app/models/user.py`, expose its metadata to Alembic, generate the first migration, apply it, and verify the `users` table was actually created in PostgreSQL.

a. Create a shared SQLAlchemy base in `app/models/__init__.py`:

`touch app/models/__init__.py`

Add:

```
"""Database model base for Triage."""
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    """Base class for all Triage database models."""
    pass
```

```
__init__.py U X
sail-triage > app > models > __init__.py > Base
1  """Database model base for Triage."""
2
3  from sqlalchemy.orm import DeclarativeBase
4
5
6  class Base(DeclarativeBase):
7      """Base class for all Triage database models."""
8
9      pass
```

b. Open `app/models/user.py` and add:

```
"""User database model for Triage."""
from datetime import datetime, timezone
from sqlalchemy import Boolean, DateTime, String, Text
from sqlalchemy.orm import Mapped, mapped_column
from app.models import Base

class User(Base):
    """Represent a Triage user account."""
    # Name of the PostgreSQL table represented by this model.
    __tablename__ = "users"
    # Primary key used internally to uniquely identify each user.
    id: Mapped[int] = mapped_column(primary_key=True)
    # Account identity fields. Usernames and email addresses must be unique.
    username: Mapped[str] = mapped_column(
        String(50),
        unique=True,
        nullable=False,
    )
    email: Mapped[str] = mapped_column(
        String(255),
        unique=True,
        nullable=False,
    )
    # Store the password hash rather than the user's actual password.
    password_hash: Mapped[str] = mapped_column(
        String(255),
        nullable=False,
    )
    # Public profile information.
    display_name: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
    )
    bio: Mapped[str | None] = mapped_column(
        Text,
        nullable=True,
    )
    # Control account permissions and whether the account can be used.
    role: Mapped[str] = mapped_column(
        String(20),
        default="user",
        nullable=False,
    )
    is_active: Mapped[bool] = mapped_column(
        Boolean,
        default=True,
        nullable=False,
    )
    # Record when the account is created and most recently updated.
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        default=lambda: datetime.now(timezone.utc),
        nullable=False,
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        default=lambda: datetime.now(timezone.utc),
        onupdate=lambda: datetime.now(timezone.utc),
        nullable=False,
    )
)
```

c. Wire Alembic to the Triage model metadata.

Open `migrations/env.py`.

Add these imports near the top:

```
from app.models import Base
from app.models.user import User
```

Then replace:

```
target_metadata = None
```

with:

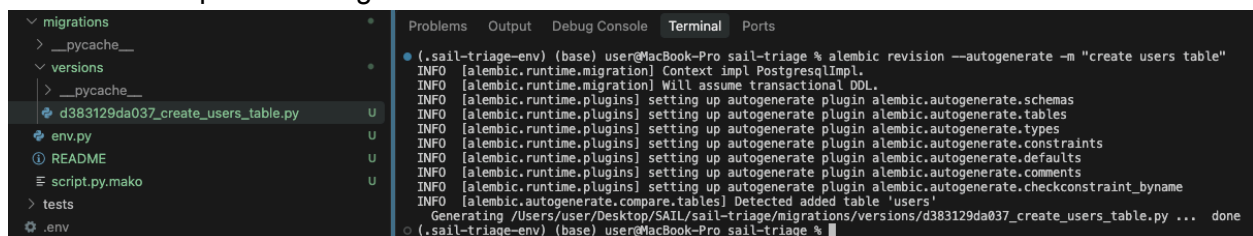
```
target_metadata = Base.metadata
```

The `User` import makes sure Alembic loads the model before inspecting `Base.metadata`.

d. Generate the first migration:

```
alembic revision --autogenerate -m "create users table"
```

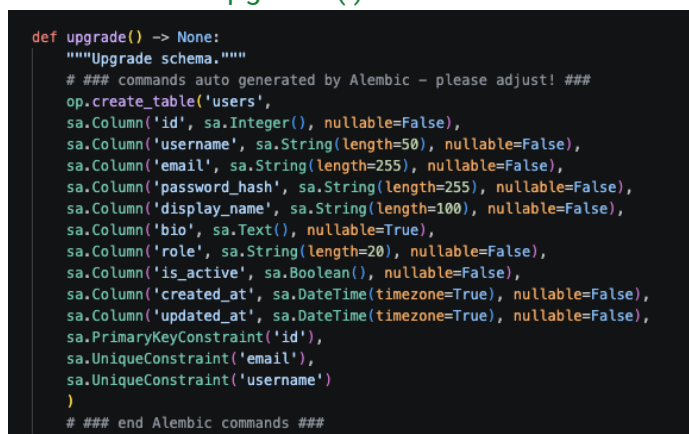
Should see output indicating Alembic detected the new `users` table.



e. Inspect the migration file created under:

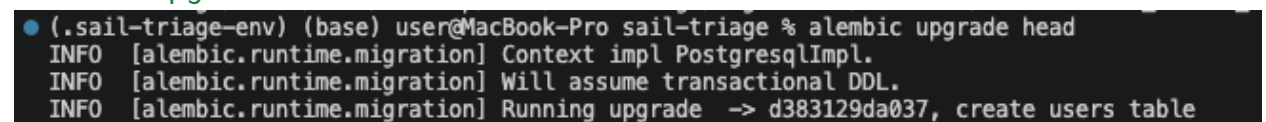
`migrations/versions/`

Confirm that its `upgrade()` function creates a `users` table with the fields from `User`.



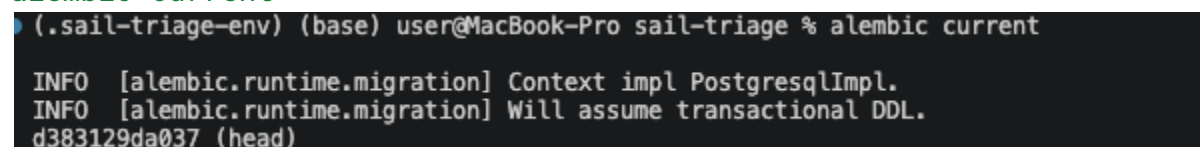
f. Apply the migration:

```
alembic upgrade head
```



g. Verify the migration state:

```
alembic current
```



It should show the revision just created and indicate (head).

h. Verify the table directly in PostgreSQL:

```
psql -U triage_app -d triage -h localhost
```

Then run:

```
\dt
```

Should see at least:

```
alembic_version
```

```
users
```

Inspect the table:

```
\d users
```

Confirm the columns include:

```
id
```

```
username
```

```
email
```

```
password_hash
```

```
display_name
```

```
bio
```

```
role
```

```
is_active
```

```
created_at
```

```
updated_at
```

```
(.sail-triage-env) (base) user@MacBook-Pro sail-triage % psql -U triage_app -d triage -h localhost
psql (17.11 (Homebrew))
Type "help" for help.

triage=> \dt
      List of relations
Schema | Name          | Type  | Owner
-----|-----|-----|-----
public | alembic_version | table | triage_app
public | users          | table | triage_app
(2 rows)

triage=> \d users
      Table "public.users"
   Column   |          Type          | Collation | Nullable |   Default
-----|-----|-----|-----|-----
id          | integer                |           | not null | nextval('users_id_seq'::regclass)
username    | character varying(50)  |           | not null |
email       | character varying(255) |           | not null |
password_hash | character varying(255) |           | not null |
display_name | character varying(100) |           | not null |
bio         | text                   |           |           |
role        | character varying(20)  |           | not null |
is_active   | boolean                |           | not null |
created_at  | timestamp with time zone |           | not null |
updated_at  | timestamp with time zone |           | not null |
Indexes:
    "users_pkey" PRIMARY KEY, btree (id)
```

Exit:

```
\q
```

Now have:

- the `User` SQLAlchemy model
- Alembic wired to the model metadata
- the first migration generated and applied
- the `users` table created in PostgreSQL
- Alembic reporting the migration at `head`

Section 9: Monitoring, Logging, & Observability

Purpose: Make the cluster and application observable enough that we can understand normal operation and diagnose failures.

Include:

- Prometheus: metrics
- Grafana: dashboards/visualization
- Loki: centralized logs
- OpenTelemetry: application telemetry/tracing
- Application instrumentation
- Node/container/Kubernetes metrics
- PostgreSQL/application metrics where useful
- Alerts for a small number of meaningful conditions
- Establish a **normal operating baseline** for comparison during Section 10

End state: We can see the health and behavior of the Pis, Kubernetes, containers, application, and database from one monitoring environment.

Section 10: Security & Infrastructure Automation

Purpose: Add security visibility and make configuration/security work reproducible rather than manually administered.

Include:

- Falco: runtime behavior detection
- Trivy: image/configuration/vulnerability scanning
- Ansible: system/configuration automation
- Terraform/OpenTofu: IaC
- Bash: appropriate system automation
- Python: more complex automation
- Security scanning incorporated into the application lifecycle
- Establish a **known security baseline**

End state: We can identify vulnerable configurations/images and suspicious runtime activity, while demonstrating repeatable infrastructure/configuration management.

Section 11: AI-Assisted Operations & Deployment Automation

This is where several pieces we've discussed come together.

Purpose: Automate application delivery and give local AI controlled access to SAIL operational information.

Include:

- GitHub Actions: CI/CD
- Helm: Kubernetes application packaging/configuration
- Argo CD: GitOps deployment
- Ollama: local model runtime
- Open WebUI: human AI interface
- **MCP:** controlled AI access to selected SAIL data/tools
- **Deterministic verification:** verify important AI claims against actual system data
- **AI provenance:** retain evidence supplied to the AI and resulting analysis
- **Human approval gates:** AI can recommend consequential remediation; you authorize it
- **Agent observability:** record AI request success/failure and latency

End state: A code change can travel through a professional automated deployment process, while the local AI can perform evidence-based analysis of the environment without being given unrestricted control.

Section 12: Event Scenarios and Investigation Workflows

The purpose of this section is to document the operational and security scenarios used to validate the SAIL environment. Each scenario will include event generation, detection mechanisms, collected evidence, AI-assisted analysis, and remediation actions.

Planned scenarios include:

Scenario	Category
Worker node failure	Infrastructure
Pod crash loop	Infrastructure
Storage utilization threshold exceeded	Infrastructure
Unauthorized shell access to a container	Security
Deployment of a vulnerable container image	Security

Excessive resource consumption by a workload	Security / Operations
Unexpected process execution within a container	Security
Service outage and recovery validation	Operations

Each scenario will document:

1. Event description
2. Event generation method
3. Detection mechanism
4. Logs and metrics collected
5. AI-generated analysis
6. Investigation process
7. Remediation actions
8. Lessons learned